Від тригера-засувки до регістра зсуву

Урок 3 — 2МАК, SST-MEC-25c — Послідовна логіка

24/09/2026

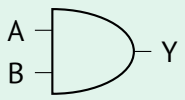
1 Повторення: основні логічні елементи

Визначення

Логічні елементи (*porte logiche*), які ми бачили досі, — це **AND**, **OR**, **NOT** та їхні комбінації **NAND** (AND + NOT на виході) і **NOR** (OR + NOT на виході). Усі п'ять дають вихід, який залежить **виключно** від поточного значення входів: така схема називається **комбінаційною** (*combinatoria*).

Приклад

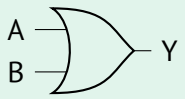
Елемент **AND**: вихід дорівнює 1 лише тоді, коли **обидва** входи дорівнюють 1.



A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

Приклад

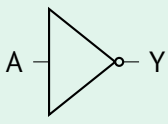
Елемент **OR**: вихід дорівнює 1, якщо **хоча б один** з двох входів дорівнює 1.



A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

Приклад

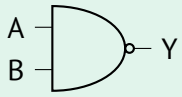
Елемент **NOT**: лише один вхід, вихід завжди протилежний входу.



A	Y
0	1
1	0

Приклад

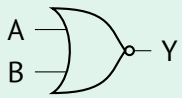
Елемент **NAND**: AND, за яким на виході йде NOT (кружечок на виході означає заперечення). Вихід **протилежний** виходу AND.



A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

Приклад

Елемент **NOR**: OR, за яким на виході йде NOT. Вихід **протилежний** виходу OR.



A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

Примітка

NAND і NOR — найуживаніші елементи в схемах високої щільності (наприклад, у мікропроцесорах): їм потрібно менше транзисторів, ніж "чистим" AND/OR (зазвичай 4 транзистори на елемент), тому вони дешевші, менші й дозволяють розмістити їх більше на тій самій площі. Сучасний **процесор** містить приблизно 10 мільярдів **транзисторів**; процесор 80-х років (наприклад, 286) мав близько 10 000 логічних елементів.

Приклад

Коли в програмі пишуть операцію на зразок `5 + 3`, не існує програми, яка "розуміє" додавання: вмикається спеціальна **комбінаційна схема**, складена з логічних елементів, яка виконує це обчислення. Простий мікроконтролер без окремої схеми для множення мусить імітувати множення послідовністю повторних додавань: тому складна операція потребує більше часу, ніж проста.

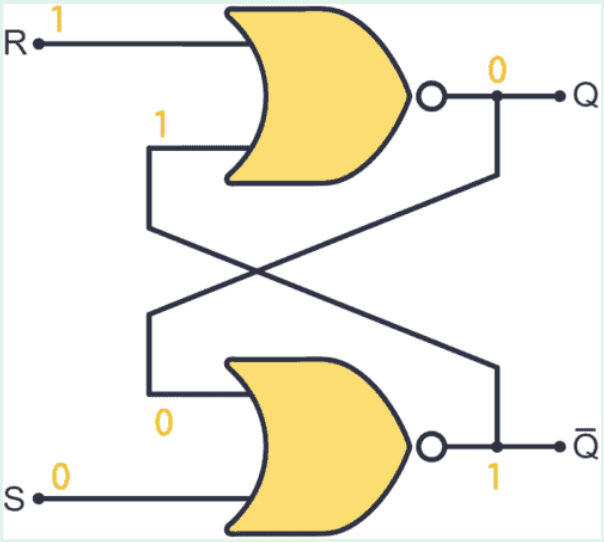
2 SR-тригер (latch SR): перша комірка пам'яті

Визначення

У **комбінаційній** схемі вихід залежить лише від поточних входів. А в **послідовній** (*sequenziale*) схемі вихід залежить також від **попереднього стану**: для цього **вихід** елемента замикають назад на його ж вхід. Таке з'єднання в коло називається **зворотним зв'язком** (*retroazione*).

Приклад

SR-тригер (Set-Reset) будують із двох перехресно з'єднаних елементів **NOR**: вихід першого елемента з'єднаний з одним входом другого, і навпаки.



При $S = 0$ і $R = 0$ схема **зберігає** останній стан: якщо раніше було $Q = 1$, залишається $Q = 1$; якщо було $Q = 0$, залишається $Q = 0$. Це і є властивість **пам'яті**.

Визначення

Таблиця істинності **SR-тригера** (на перехресних NOR):

S	R	Q	$\overline{Q}$
0	0	зберігає	зберігає
1	0	1	0
0	1	0	1
1	1	*заборонений стан*	

Увага

Комбінація $S = 1, R = 1$ **заборонена**: вона примусила б **обидва виходи стати 0**, порушуючи співвідношення $Q = \overline{\overline{Q}}$, яке схема завжди має виконувати (два виходи мають бути **завжди запереченням один одного**). Її завжди треба уникати.

Примітка

Назви входів не випадкові: **Set** означає "встановити", а **Reset** — "скинути". Подаючи $S = 1$, встановлюємо (зберігаємо) 1 на виході; подаючи $R = 1$, **встановлюємо** (стираємо) 0. При $S = R = 0$ нової команди немає, і схема лише зберігає останнє встановлене значення.

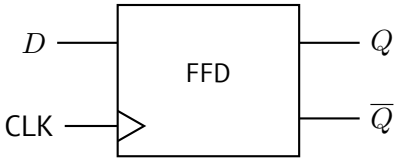
Примітка

SR-тригер був одним із найперших пристроїв **твердотільної пам'яті**. До того комп'ютери використовували механічні системи на зразок **пам'яті на феритових осердях** (*core memory*): сітка горизонтальних і вертикальних дротів із маленьким кільцем із намагнічуваного матеріалу на кожному перетині; наявність чи відсутність намагнічення в цьому кільці означала **біт**. Така шафа завбільшки з холодильник містила лише кілька кілобайтів пам'яті.

3 D-тригер (flip-flop D)

Визначення

Існують складніші варіанти тригера-засувки, які не дають потрапити в **заборонений стан**. Той, яким ми користуватимемося до кінця року, — це **D-тригер** (*flip-flop di tipo D*, скорочено **FFD**): **елемент пам'яті** з двома входами (**D**, дані; **CLK**, тактовий сигнал — clock) і двома виходами (Q, $\overline{Q}$).



Приклад

D-тригер працює як фотоапарат: вхід D може вільно змінюватися з часом, але це ніяк не впливає на вихід. Лише в мить, коли тактовий сигнал переходить з низького рівня на високий (**передній фронт**, *fronte di salita*), тригер "**фотографує**" значення, яке D має саме в цю мить, і передає його на вихід Q, де воно залишається до наступного переднього фронту.

Увага

Тригер реагує лише на передній фронт тактового сигналу, ніколи на задній фронт (коли clock переходить з високого рівня на низький) і ніколи на саме значення clock. **Задній фронт** (*fronte di discesa*) не оновлює вихід.

4 Асинхронний лічильник

Визначення

Якщо з'єднати кілька D-тригерів **каскадом** – вихід $\overline{Q}$ кожного з'єднаний з його ж входом D, а **clock** кожного ступеня керується виходом $\overline{Q}$ **попереднього** ступеня (лише перший тригер отримує "справжній" clock) – отримаємо **асинхронний лічильник**: на кожному **активному фронті** керувального сигналу кожен тригер переходить до наступного значення, і зміна поширюється каскадом від одного ступеня до наступного. Він називається **асинхронним**, бо тригери змінюються не **всі** в **одну й ту саму мить**: кожен чекає, поки зміниться попередній, і від ступеня до ступеня з'являється невелика **затримка поширення**.

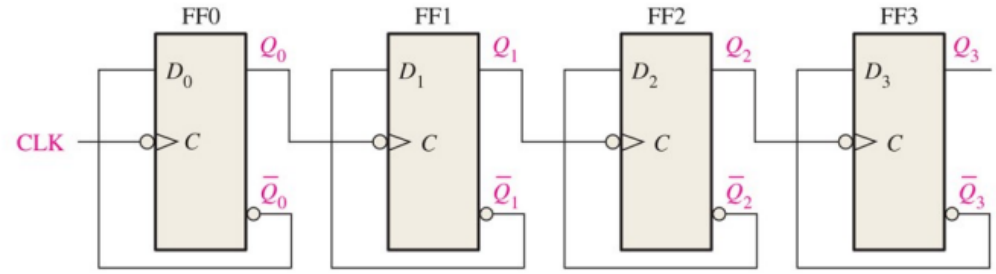
Приклад

Асинхронний **4-бітний** лічильник із 4 D-тригерів (**FF0, FF1, FF2, FF3**), кожен із яких **спрацьовує на задньому фронті**. Зовнішній **CLK** керує **лише FF0**; далі вихід $\overline{Q}$ кожного ступеня стає **clock наступного ступеня**. У кожному ступені $\overline{Q}$ також з'єднаний з його ж D (як у лічильнику, розглянутому в теорії), тому кожен тригер перемикається на кожному своєму фронті clock.

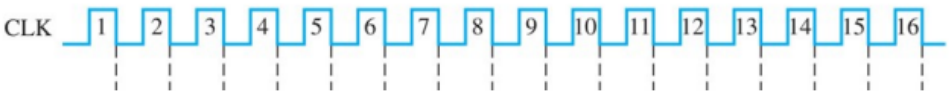
A 4-bit asynchronous binary counter is shown below. Each D flip-flop is negative edge-triggered.

(a) Develop a timing diagram showing the Q output of each flip-flop.

(b) Redesign the circuit for a decade counter.



(a)



Виходи Q_0, Q_1, Q_2, Q_3 утворюють лічене двійкове число (Q_0 — молодший біт): на кожному задньому фронті зовнішнього CLK спрацьовує FF0 і Q_0 змінюється на протилежне, отже, змінюється і $\overline{Q}_0$; коли $\overline{Q}_0$ має свій задній фронт (тобто коли Q_0 переходить з 0 в 1), спрацьовує FF1 і змінюється Q_1; і так далі каскадом на наступних ступенях.

Приклад

Починаючи з 0000, кожен такт збільшує лічбу: $0000 \to 0001 \to 0010 \to 0011 \to 0100 \to \dots$ Перший вихід (Q_0) змінюється на кожному такті, другий (Q_1) — кожні 2 такти, третій (Q_2) — кожні 4 такти, четвертий (Q_3) — кожні 8 тактів: кожен ступінь **удвічі сповільнює** зміну порівняно з попереднім – точно як у синхронному лічильнику, але тут зміна **поширюється від ступеня до ступеня**, а не відбувається одночасно.

Примітка

Щоб прочитати лічене число, треба вміти переводити між **основою 2** і **основою 10**. Діє **позиційний запис**: крайній правий біт важить 2^0, наступний 2^1, далі 2^2, 2^3 і так далі (пам'ятаючи, що будь-яке число **в степені 0** дорівнює 1). Наприклад, двійкове число 10010 переводять так:

$$1 \cdot 2^4 + 0 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 16 + 0 + 0 + 2 + 0 = 18$$

Для зворотного напрямку (з **десяткового в двійкове**) **послідовно ділять на 2**, записуючи остачі, а потім читають остачі від останньої до першої: ділячи 18 на 2 кілька разів поспіль (18, 9, 4, 2, 1), отримуємо остачі $0, 1, 0, 0, 1$, які, прочитані у зворотному порядку, дають 10010.

Примітка

Лічильник такого типу лежить в основі поняття **program counter** (лічильник команд) у **мікропроцесорах**: регістра, який стежить, який рядок коду треба виконати. Машина виконує команди по одній, і на кожному кроці program counter переходить до наступної. Швидкість, з якою просувається program counter, пов'язана з **тактовою частотою** процесора: чим швидший clock, тим швидше виконуються команди. (Справжні program counter у мікропроцесорах майже завжди **синхронні**, щоб уникнути **затримок поширення** асинхронної версії – але принцип двійкової лічби той самий.)

Та сама схема, якщо подивитися на неї інакше, є також **дільником частоти**: кожен ступінь удвічі зменшує частоту сигналу, який нею керує. **Цифровий годинник**, наприклад, використовує ланцюжок таких дільників, щоб перетворити дуже швидкий і точний clock на імпульс, який з'являється рівно раз на секунду.

Синхронний лічильник, для порівняння: існує також варіант, у якому **всі тригери мають один і той самий** тактовий сигнал (а не керують один одним каскадом): усі ступені змінюються в одну мить, без маленьких затримок поширення асинхронної версії. Саме його обирають у реальних швидкісних схемах, але він потребує трохи складнішої логіки керування, щоб вирішувати, коли кожен ступінь має збільшуватися.

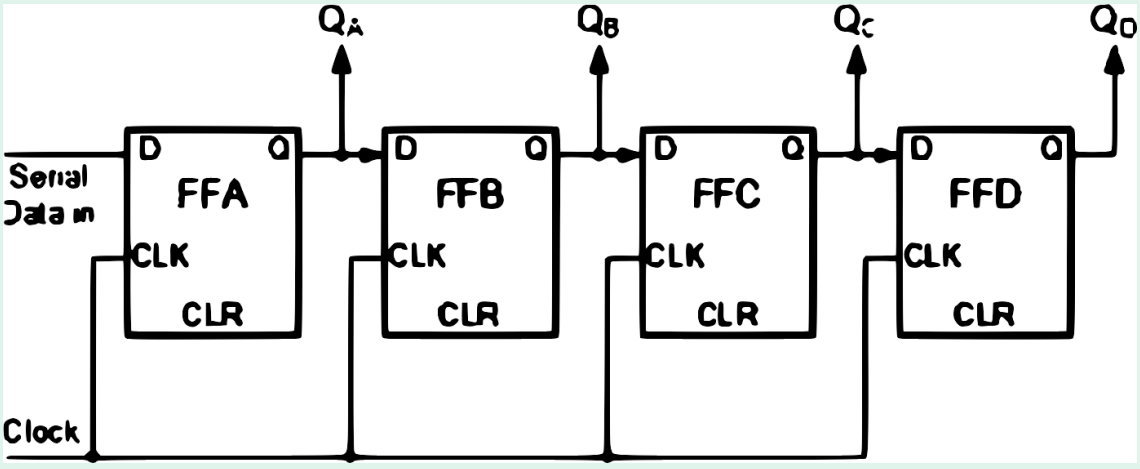
5 Регістр зсуву (shift register)

Визначення

Якщо змінити з'єднання між D-тригерами в каскаді – вихід Q кожного з'єднаний із **входом** D **наступного**, а не $\overline{Q}$ зі своїм D – отримаємо **регістр зсуву** (*registro a scorrimento*, *shift register*): пристрій, який на кожному такті **просуває по одному біту** вздовж ланцюжка.

Приклад

Повний **4-бітний** регістр зсуву, як його накреслено на дошці: чотири D-тригери в каскаді (FFA, FFB, FFC, FFD). Вихід Q кожного стає входом D **наступного** (а не свого, як було в лічильнику), і всі **мають один і той самий** сигнал **Clock**.



Увесь ланцюжок має вхід **Serial Data in** на першому тригері (FFA) і чотири виходи

Q_A, Q_B, Q_C, Q_D, які можна **читати** **паралельно**.

Приклад

Корисна аналогія: **картонна** **трубка**, у яку по одній вкладають кульки двох різних кольорів (білі й чорні, вони означають біти 1 і 0). Щоразу, коли з одного кінця вкладають нову кульку, усі інші просуваються на одну позицію. Дивлячись на трубку ззовні, бачиш **усі** **кульки** **одночасно**, хоча їх вкладали по одній, послідовно.
Саме це і робить **shift** **register**: дані **входять** **послідовно** (по одному біту, одним проводом SER), а прочитати їх можна всі разом **паралельно** (кількома проводами одночасно, виходи Q_A, Q_B, Q_C, Q_D). Тому говорять про перетворення **послідовний** → **паралельний**.

6 Мікросхема 74HC595

Визначення

Промисловий компонент, яким ми користуватимемося для своїх схем, — **74HC595**, **8-бітний** регістр зсуву з виходами **на** **3** **стани**. Він належить до сімейства мікросхем **74** (**комерційне** застосування, вужчий діапазон температур), на противагу сімейству **54** (військове застосування, ширший діапазон температур). Позначення **HC** вказує на **технологію** **виготовлення** **швидкісний** **CMOS** (*High-speed CMOS*).

Примітка

Виходи **на** **3** **стани** (*three-state* або *tri-state*) можуть мати, крім звичайних рівнів логічного нуля (0) і одиниці (1), **третій** **стан**: **високий** **імпеданс** (електрично не під'єднано). Це дозволяє під'єднати кілька виходів 3-state до одного проводу без ризику **короткого** **замикання**, бо лише один пристрій одночасно "говорить" у провід, а інші залишаються від'єднаними.

Приклад

Основні технічні дані 74HC595 (з datasheet Texas Instruments):

- **напруга** **живлення**: від 2 В до 6 В (його спокійно живлять від 5 В, що відповідає стандартній логіці TTL/CMOS);
- **споживання** **в** **режимі** **малої** **потужності**: приблизно 80 μA (0,00008 А), надзвичайно мало.

Увага

Якщо перевищити **максимальну** **напругу**, зазначену в datasheet (6 В), можна **спалити** **компонент**. Якщо живити його недостатньою напругою (наприклад, 1 В), це його не пошкодить, але він просто **не** **працюватиме**.

6.1 Корпус: PTH чи SMD

Визначення

Електронні компоненти поділяють за типом монтажу на плату:

- **PTH** (*Pin Through Hole*, з наскрізним отвором): **виводи проходять крізь отвір** у платі, і їх паяють з протилежного боку. Це тип компонентів, якими досі користувалися в лабораторії (наприклад, на макетній платі, *breadboard*).
- **SMD** (*Surface Mounting Device*, поверхневий монтаж): виводи загнуті "**крилом чайки**", їх просто кладуть і припаюють на поверхню плати, **без отворів**.

Примітка

Перевага SMD — **місце**: не треба відводити частину плати під **наскрізні отвори**, тому можна робити **компактніші й щільніші** схеми. Недолік у тому, що **паяння SMD** потребує іншої техніки, ніж PTH, делікатнішої через маленький розмір виводів.

Приклад

Поширені корпуси (*package*) для 74HC595:

- **DIP** (*Dual Inline Package*): два ряди виводів, типовий для PTH. З керамічним корпусом говорять про **CDIP**, з пластиковим — про **PDIP** (найпоширеніший у наших лабораторіях).
- SOIC, **SOP**: корпуси **SMD**, менші, з кроком між виводами 1,27 мм (проти стандартного кроку 2,54 мм/100 mil у PTH). Ще компактніші варіанти, як-от **SSOP**/**TSSOP**, мають крок аж до 0,65 мм.

Точне позначення компонента для замовлення залежить від корпусу: наприклад, `SN54HC595J` для **керамічної** версії, `74HC595D` для версії **SOIC**.

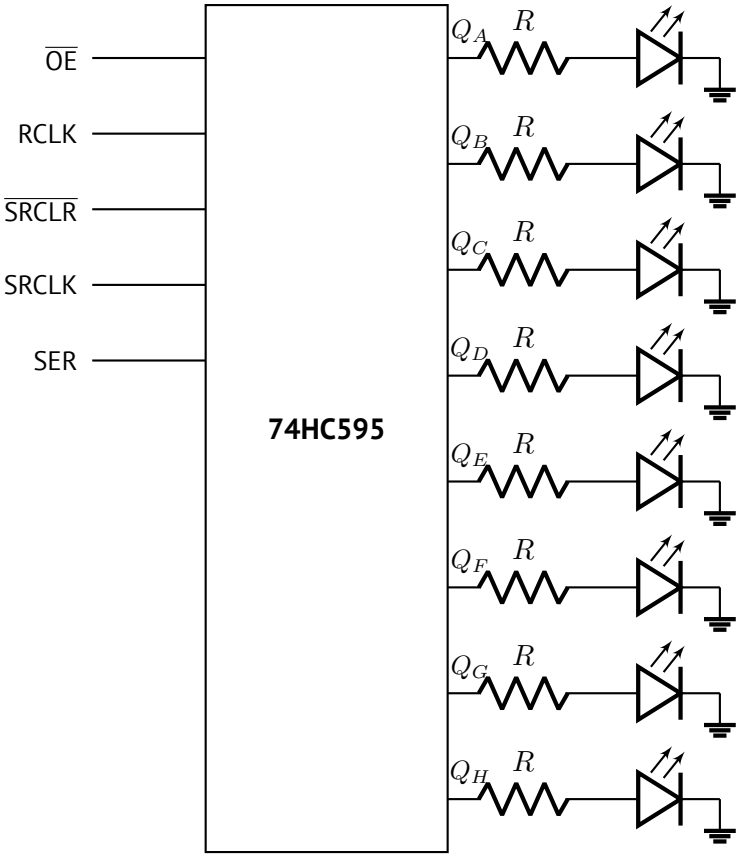
6.2 Розташування виводів 74HC595

Визначення

Керувальні виводи 74HC595 за **офіційним datasheet** (Texas Instruments):

- **SER** (вивід 14): **послідовний вхід даних**, куди біти подають по одному – відповідає входу D нашого D-тригера.
- **SRCLK** (вивід 11): **clock внутрішнього регістра зсуву** – кожен **передній фронт** просуває на одну позицію біт, щойно поданий на SER, точно як у схемі на D-тригерах вище.
- **RCLK** (вивід 12, *Register Clock*, його також називають *latch clock*): **другий** регістр, окремий від внутрішнього регістра зсуву, який **копіює всі 8 бітів** на виходи лише тоді, коли отримує свій **передній фронт**.
- **SRCLR** (вивід 10, *Shift Register Clear*, інверсний): скидає весь внутрішній регістр зсуву. У нашому використанні він завжди під'єднаний до $+5$ В (високий рівень, "не активний": оскільки він інверсний, він спрацьовує лише тоді, коли на нього подають низький рівень).

- **OE** (вивід 13, *Output Enable*, інверсний): вмикає або вимикає виходи на 3 стани. Він завжди під'єднаний до GND (низький рівень, "завжди ввімкнено").



Примітка

Причина, чому 74HC595 має **два** окремі регістри (регістр зсуву, яким керує SRCLK, і **вихідний регістр**, яким керує RCLK), — уникнути небажаного ефекту: якби виходи оновлювалися на кожному такті SRCLK, поки біти подають по одному, на під'єднаних світлодіодах було б видно **зсув** даних, а не миттєву й чисту зміну. Завдяки розділенню двох clock усі 8 бітів спочатку завантажують у регістр зсуву (SRCLK), і лише наприкінці одним тактом RCLK їх **виводять усі разом** на виходи: зміна виглядає миттєвою й "асинхронною" щодо завантаження.

Приклад

Послідовність команд, щоб керувати 74HC595 з **мікроконтролера** (наприклад, Arduino), для кожного з 8 бітів, які треба передати:

1. Записати значення біта (0 або 1) на вивід SER.
2. **Створити передній фронт** на SRCLK (високий рівень, потім низький): це **просуває біт** у внутрішньому регістрі зсуву.
3. Повторити для всіх 8 бітів, від першого до останнього.
4. Лише наприкінці, коли всі 8 бітів завантажено, **створити** один **передній фронт** на RCLK: це **виводить усі дані** на виходи одночасно, без **видимого зсуву**.

У нашому використанні SRCLR завжди під'єднаний до $+5$ В, а OE — до GND: **єдині** **три** **виводи**, якими мікроконтролер справді має керувати, — це SER, SRCLK і RCLK.

7 Що запам'ятати з цього уроку

- У **комбінаційній** схемі вихід залежить лише від поточних входів; у **послідовній** — також від **попереднього** **стану**, завдяки **зворотному** **зв'язку**.
- **SR-тригер** (два перехресні NOR) — перша комірка пам'яті: він зберігає стан при $S = R = 0$, але має **заборонену** комбінацію ($S = R = 1$).
- **D-тригер** розв'язує цю проблему: він оновлює вихід **лише** на **передньому** **фронті** clock і працює як **фотоапарат** для входу D.
- З'єднавши D-тригери каскадом з $\overline{Q} \rightarrow D$, отримаємо **лічильник** (*асинхронний*, де кожен ступінь керується $\overline{Q}$ попереднього, або *синхронний*, з усіма CLK спільними) – основу ***program counter*** і **дільників** **частоти**; з $Q \rightarrow D$ і спільним для всіх ступенів CLK отримаємо натомість **регістр** **зсуву** (перетворення послідовний→паралельний).
- **74HC595** — **8-бітний** регістр зсуву з виходами 3-state, яким керують **три** **сигнали**: **SER** (**дані**), **SRCLK** (**просуває** **регістр** **зсуву**), **RCLK** (**виводить** **результат** на виходи, без видимого зсуву).
- Компоненти розрізняють за монтажем **PTH** (в отвір, наприклад DIP) або **SMD** (поверхневий, наприклад SOIC/SOP), з **різними** **корпусами** **й** **позначеннями** залежно від вибору.

Глосарій: італійські слова, які вживають на уроці

Italiano	**Українською**
porta logica	логічний елемент
ingresso, uscita	вхід, вихід
tabella di verità	таблиця істинності
circuito combinatorio / sequenziale	комбінаційна / послідовна схема
retroazione	зворотний зв'язок
latch, flip-flop	тригер-засувка, тригер
stato non ammesso	заборонений стан
fronte di salita / di discesa	передній / задній фронт
contatore	лічильник
divisore di frequenza	дільник частоти
registro a scorrimento	регістр зсуву
alta impedenza	високий імпеданс
alimentazione	живлення
piedino, piedinatura	вивід, розташування виводів